操作系统实验报告

第二次实验：uC/OS-II 时间片轮转调度器实现

山东大学 网络空间安全学院 张宇杰

2026 年 4 月 10 日

目录

1 实验概述 **2**

2 任务一：单队列时间片轮转调度 **2**

2.1 设计思路 2

2.2 就绪队列实现（os_core.c） 3

2.2.1 OS_InitRdyQueue——初始化就绪队列 3

2.2.2 OSRdyQueueIn——任务入队（队尾插入） 3

2.2.3 OSRdyQueueOut——队头出队 4

2.3 时钟中断处理（OSTimeTick） 5

2.4 调度器核心修改（OS_SchedNew） 5

2.5 新建任务时入队（OS_TCBInit） 6

2.6 任务删除时出队（os_task.c） 7

2.7 扩展系统调用——OSTaskDel（user_asm.s） 7

2.7.1 导入/导出声明 7

2.7.2 SVC 分派 7

2.7.3 OSTaskDel 调用桩 7

2.7.4 用户侧包装函数 8

2.8 测试用例（main.c） 8

3 任务二：分组时间片轮转调度（可选） **9**

3.1 设计思路 9

3.2 全局变量修改（ucos_ii.h） 9

3.3 就绪队列原语修改（os_core.c） 10

3.3.1 OS_InitRdyQueue——批量清零 10

3.3.2 OSRdyQueueIn——按组入队 10

3.3.3 OSRdyQueueOut——按组出队 11

3.4 调度器修改（OS_SchedNew） 12

3.5 任务删除适配（os_task.c） 13

3.6 测试用例扩展（main.c） 13

4 总结 **14**

1 实验概述

本实验在 C/OS-II 实时操作系统的基础上，分两个任务实现时间片轮转（Round-Robin）调度机制：

- **任务一**：为同一优先级的所有就绪任务实现单队列的时间片轮转调度，使相同优先级的任务能够公平地共享 CPU。
- **任务二（可选）**：在任务一的基础上扩展为分组时间片轮转调度——每 8 个连续优先级为一组，共 8 组；组内实施轮转调度，组间仍按优先级调度。

实验涉及的主要文件如下表所示：

文件	路径	主要修改内容
`main.c`	工程根目录	新增测试任务及其入口函数
`os_core.c`	src/ucos_ii/src/	就绪队列、调度器核心逻辑
`os_task.c`	src/ucos_ii/src/	任务删除时出队处理
`ucos_ii.h`	src/ucos_ii/src/	就绪队列全局变量声明
`user_asm.s`	工程根目录	新增 `OSTaskDel` 系统调用

2 任务一：单队列时间片轮转调度

2.1 设计思路

原始 C/OS-II 采用纯优先级抢占调度：每次调度总是选取优先级最高的就绪任务运行，相同优先级的任务无法共享 CPU。为了实现时间片轮转，需要在不破坏优先级抢占语义的前提下，对处于”当前最高优先级”的多个就绪任务维护一条 FIFO 就绪队列，并在每次时钟中断时递减当前任务的剩余时间片，时间片耗尽时将该任务移到队尾，从队头取下一个任务投入运行。

整体修改分为以下几个层次：

1. 在 `os_core.c` 中实现就绪队列的三个原语:`OSRdyQueueIn`、`OSRdyQueueOut`、`OS_InitRdyQueue`。
2. 在 `OSTimeTick` 中递减当前任务的剩余时间片，并在任务从阻塞态回到就绪态时将其入队。
3. 修改 `OS_SchedNew`，使时间片耗尽时执行轮转逻辑。
4. 在 `OS_TCBInit`（`OSTaskCreate` 的子程序）中对新建任务入队。
5. 在 `OSTaskDel` 中对被删除的任务出队。

6. 扩展 main.c 增加第三个测试任务，并通过 user_asm.s 暴露 OSTaskDel 的系统调用接口。

2.2 就绪队列实现（os_core.c）

就绪队列是一条带头尾指针的双向链表,利用 TCB 中新增的 OSRdyTCBNext 和 OSRdyTCBPrev 两个指针穿串。三个全局变量（OSRdyTCBQueueFront、OSRdyTCBQueueRear、OSRdyTCBQueueNum）分别记录队头、队尾和队列长度，均在 ucos_ii.h 中以 OS_EXT 宏声明，保证跨文件可见。

2.2.1 OS_InitRdyQueue——初始化就绪队列

在系统启动时（OSInit 内）被调用，将队头、队尾置 NULL，队列长度归零。

```
+void OS_InitRdyQueue(void)
+{
+    /* 队列为空：头尾指针都指向 NULL */
+    OSRdyTCBQueueFront = (OS_TCB *)0;
+    OSRdyTCBQueueRear  = (OS_TCB *)0;
+
+    /* 队列中没有任务 */
+    OSRdyTCBQueueNum   = 0;
+}
```

Listing 1: os_core.c —就绪队列初始化

2.2.2 OSRdyQueueIn——任务入队（队尾插入）

当一个任务需要进入就绪队列时（新建、从阻塞态恢复、时间片耗尽后重新排队），调用此函数。

- 空闲任务（OS_TASK_IDLE_PRIO）不参与轮转，直接返回。
- 重置该任务的剩余时间片为 OS_SCHED_QUANTUM_MAX。
- 若队列为空，则新节点同时成为队头和队尾；否则链接到队尾并更新尾指针。

```
+void OSRdyQueueIn(OS_TCB *ptcb)
+{
+    if (ptcb->OSTCBPrio == OS_TASK_IDLE_PRIO)
+        return;                    /* 空闲任务不进入轮转队列 */
+
+    ptcb->quantum      = OS_SCHED_QUANTUM_MAX;   /* 重置时间片 */
```

```
+    ptcb->OSRdyTCBNext = (OS_TCB *)0;
+    ptcb->OSRdyTCBPrev = (OS_TCB *)0;
+
+    if (OSRdyTCBQueueNum == 0) {
+        /* 队列为空，成为唯一节点 */
+        OSRdyTCBQueueFront = ptcb;
+        OSRdyTCBQueueRear  = ptcb;
+    } else {
+        /* 插入队尾 */
+        ptcb->OSRdyTCBPrev               = OSRdyTCBQueueRear;
+        OSRdyTCBQueueRear->OSRdyTCBNext = ptcb;
+        OSRdyTCBQueueRear                = ptcb;
+    }
+    OSRdyTCBQueueNum++;
+}
```

Listing 2: os_core.c —任务入队

2.2.3 OSRdyQueueOut——队头出队

每次轮转时，从队头取出下一个将要运行的任务。若队列为空则返回 NULL；若只有一个节点则队列归空；否则队头指针后移并清除新队头的前向指针。

```
+OS_TCB *OSRdyQueueOut(void)
+{
+    OS_TCB *ptcb;
+
+    if (OSRdyTCBQueueFront == (OS_TCB *)0)
+        return (OS_TCB *)0;          /* 队列为空 */
+
+    ptcb = OSRdyTCBQueueFront;       /* 取出队头 */
+
+    if (OSRdyTCBQueueNum == 1) {
+        /* 只剩一个节点，队列变空 */
+        OSRdyTCBQueueFront = (OS_TCB *)0;
+        OSRdyTCBQueueRear  = (OS_TCB *)0;
+    } else {
+        /* 队头后移，清除新队头的前向指针 */
+        OSRdyTCBQueueFront               = ptcb->OSRdyTCBNext;
+        OSRdyTCBQueueFront->OSRdyTCBPrev = (OS_TCB *)0;
+    }
```

```
+     /* 清空出队节点的链表指针，避免悬空指针 */
+     ptcb->OSRdyTCBNext = (OS_TCB *)0;
+     ptcb->OSRdyTCBPrev = (OS_TCB *)0;
+     OSRdyTCBQueueNum--;
+     return ptcb;
+}
```

Listing 3: os_core.c —队头出队

2.3 时钟中断处理（OSTimeTick）

OSTimeTick 在每次 SysTick 中断时被调用，负责维护时间相关的状态。需要在此完成两件事：

递减当前任务的剩余时间片

```
 #if OS_SCHED_ROUND_ROBIN_EN > 0
+     if (OSTCBCur->quantum > 0) {
+         OSTCBCur->quantum--;    /* 每个时钟周期减 1; 减到 0 时触发轮转 */
+     }
 #endif
```

Listing 4: os_core.c —递减当前任务剩余时间片

阻塞态任务唤醒时入队 原有代码已将解除阻塞的任务通过 OSRdyGrp/OSRdyTbl 标记为就绪，但轮转调度还需将其加入就绪队列。条件判断由框架提供：若该任务此前不在就绪组/就绪位图中（即原来处于非就绪态），则现在入队。

```
 #if OS_SCHED_ROUND_ROBIN_EN > 0
     if ((OSRdyGrp & ptcb->OSTCBBitY) == 0 ||
         (OSRdyTbl[ptcb->OSTCBY] & ptcb->OSTCBBitX) == 0)
+         OSRdyQueueIn(ptcb);    /* 从阻塞态恢复的任务重新入就绪队列 */
 #endif
```

Listing 5: os_core.c —阻塞转就绪时入队

2.4 调度器核心修改（OS_SchedNew）

OS_SchedNew 负责决定下一个运行的任务（将结果写入全局变量 OSPrioHighRdy）。在轮转模式下，当当前任务时间片耗尽时，需将其移到队尾并从队头选取下一个任务。

逻辑分两种情形：

1. **系统刚启动（OSTCBCur == 0）**：尚无”当前任务”，直接按位图找最高优先级任务。
2. **当前任务时间片耗尽（quantum == 0）**：先出队再入队（实现”移到队尾”），然后从队头取新任务的优先级；若队列为空（无同优先级可轮转的任务），回退到位图查找。

```
 #if OS_SCHED_ROUND_ROBIN_EN > 0
+    if (OSTCBCur == 0 | OSTCBCur->quantum == 0)
+    {
+        if (OSTCBCur != 0) {
+            OSRdyQueueOut();          /* 把当前任务从队头移走 */
+            OSRdyQueueIn(OSTCBCur);   /* 重新插入队尾，完成轮转 */
+        }
+        /* 选取下一个任务：优先从队列头部取，队列空则退回位图调度 */
+        if (OSRdyTCBQueueFront != (OS_TCB *)0) {
+            OSPrioHighRdy = OSRdyTCBQueueFront->OSTCBPrio;
+        } else {
+            INT8U y;
+            y = OSUnMapTbl[OSRdyGrp];
+            OSPrioHighRdy = (INT8U)((y << 3) + OSUnMapTbl[OSRdyTbl[y]]);
+        }
+    }
 #endif
```

Listing 6: os_core.c —OS_SchedNew 轮转逻辑

2.5 新建任务时入队（OS_TCBInit）

OS_TCBInit 由 OSTaskCreate 调用，在 TCB 初始化完成后将新任务加入就绪队列，使其能够参与后续的轮转调度。

```
 #if OS_SCHED_ROUND_ROBIN_EN > 0
-    // your code:  put ptcb into queue of ready TCBs
+    OSRdyQueueIn(ptcb);   /* 新建任务加入轮转就绪队列 */
 #endif
```

Listing 7: os_core.c —新建任务入就绪队列

2.6 任务删除时出队（os_task.c）

当一个任务主动调用 OSTaskDel 删除自身时，必须将其从就绪队列中移除，否则轮转队列中将保留悬空的 TCB 指针，导致非法内存访问。

```
 #if OS_SCHED_ROUND_ROBIN_EN > 0
+    OSRdyQueueOut();    /* 从轮转就绪队列中移除当前任务 */
+    OSTCBCur = 0;       /* 清空当前任务指针，触发重新调度 */
 #endif
```

Listing 8: os_task.c —任务删除时出队

将 OSTCBCur 置零后，下一次 OS_SchedNew 进入 OSTCBCur == 0 分支，系统将正确地从队列或位图中选取新任务。

2.7 扩展系统调用——OSTaskDel（user_asm.s）

为了让运行在非特权模式下的用户任务能够调用 OSTaskDel，需要在汇编层面扩展 SVC（系统调用）分派表，分配新的调用号 0x06。

2.7.1 导入/导出声明

```
 IMPORT OSTaskCreate
+IMPORT OSTaskDel
 EXPORT syscall_OSTaskCreate
+EXPORT syscall_OSTaskDel
```

Listing 9: user_asm.s —新增 OSTaskDel 导入导出

2.7.2 SVC 分派

在 SVC_Handler 中增加对调用号 0x06 的判断：

```
     CMP    R0, #0x05
     BEQ    OSTaskCreate_call
+    CMP    R0, #0x06
+    BEQ    OSTaskDel_call
     BX     LR
```

Listing 10: user_asm.s —SVC 分派新增 OSTaskDel

2.7.3 OSTaskDel 调用桩

```
+OSTaskDel_call
+     LDR    R0, [R2]        ; 从用户栈加载参数（优先级号）
+     MOV    R1, #0xFFFF
+     AND    R0, R1          ; 截取低 16 位，确保参数合法
+     PUSH   {LR}
+     BL     OSTaskDel       ; 调用内核函数
+     POP    {LR}
+     BX     LR
```

Listing 11: user_asm.s —OSTaskDel 调用桩实现

2.7.4 用户侧包装函数

```
+syscall_OSTaskDel
+     SWI 0x06     ; 触发 SVC，调用号 0x06
+     BX  LR
```

Listing 12: user_asm.s —syscall_OSTaskDel 用户包装

2.8 测试用例（main.c）

在原有两个任务的基础上新增 Task 3，三个任务具有相同优先级，用于验证时间片轮转是否正常工作。每个任务循环打印 9 次后调用 `syscall_OSTaskDel` 退出。

```
 static OS_STK task2_stk[TASK1_STK_SIZE];
+static OS_STK task3_stk[TASK1_STK_SIZE];
 void task1(void *p_arg);
 void task2(void *p_arg);
+void task3(void *p_arg);
+INT8U  syscall_OSTaskDel (INT8U prio);
```

Listing 13: main.c —新增第三个测试任务

```
+     OSTaskCreate(task3, (void *)0,
+         &task3_stk[TASK3_STK_SIZE - 1], TASK3_PRIO);

+static void task1(void *p_arg)
+{
+     for (int i = 1; i < 10; i++) {
+         syscall_print_str("Hello from Task 1!\n");
+     }
+     syscall_OSTaskDel(OS_PRIO_SELF);  /* 完成后删除自身 */
```

```
+}

 /* task2 和 task3 结构类似，分别打印 Task 2 / Task 3 标识后自删 */
```

Listing 14: main.c —在 main() 中创建 Task 3 并修改各任务逻辑

在上述测试中，三个任务共享同一优先级，预期输出应交替出现 Task 1 / Task 2 / Task 3，每轮切换恰好在一个时间片（OS_SCHED_QUANTUM_MAX 个 tick）之后发生，从而验证轮转调度的正确性。

经调试，调度器正确工作，三个 task 正确轮换，在 task1 退出后，剩余两个 task 也正确轮换：

3 任务二：分组时间片轮转调度（可选）

3.1 设计思路

任务二在任务一的基础上将轮转调度的范围限制在同一优先级组内：C/OS-II 将 0–63 号优先级按每 8 个一组分为 8 组（组号 = 优先级 » 3）。组间依然按优先级高低抢占，组内各任务轮转。

关键改动是将任务一中的单条就绪队列（OSRdyTCBQueueFront 等）扩展为 8 条队列（下标即组号），并修改 OSRdyQueueIn / OSRdyQueueOut / OS_SchedNew 的相应逻辑。

3.2 全局变量修改（ucos_ii.h）

将单条队列的头尾指针和计数器替换为各含 8 个元素的数组：

```
 #if OS_SCHED_ROUND_ROBIN_EN > 0
-OS_EXT   OS_TCB            *OSRdyTCBQueueFront;
-OS_EXT   OS_TCB            *OSRdyTCBQueueRear;
-OS_EXT   INT8U              OSRdyTCBQueueNum;
```

```
+OS_EXT   OS_TCB            *OSRdyTCBQueueFrontTbl[8];   /* 每组队头 */
+OS_EXT   OS_TCB            *OSRdyTCBQueueRearTbl[8];    /* 每组队尾 */
+OS_EXT   INT8U              OSRdyTCBQueueNumTbl[8];     /* 每组队列长度 */
 #endif
```

Listing 15: ucos_ii.h —就绪队列全局变量扩展为 8 组

3.3 就绪队列原语修改（os_core.c）

3.3.1 OS_InitRdyQueue——批量清零

使用 OS_MemClr 对三个数组整体清零，比逐项赋值更简洁：

```
 void OS_InitRdyQueue(void)
 {
+    OS_MemClr((INT8U *)&OSRdyTCBQueueFrontTbl[0], sizeof(OSRdyTCBQueueFrontTbl));
+    OS_MemClr((INT8U *)&OSRdyTCBQueueRearTbl[0],  sizeof(OSRdyTCBQueueRearTbl));
+    OS_MemClr((INT8U *)&OSRdyTCBQueueNumTbl[0],   sizeof(OSRdyTCBQueueNumTbl));
 }
```

Listing 16: os_core.c —分组队列初始化

3.3.2 OSRdyQueueIn——按组入队

通过 TCB 中已有的字段 OSTCBY（= 优先级 » 3）确定所属组号，后续逻辑与任务一完全相同，只是操作对象从单条队列变为对应组的队列：

```
 void OSRdyQueueIn(OS_TCB *ptcb)
 {
+    INT8U grp;
     if (ptcb->OSTCBPrio == OS_TASK_IDLE_PRIO)
         return;
+    grp = ptcb->OSTCBY;   /* 优先级组号 = 优先级 >> 3 */

+    ptcb->quantum      = OS_SCHED_QUANTUM_MAX;
+    ptcb->OSRdyTCBNext = (OS_TCB *)0;
```

```
+    ptcb->OSRdyTCBPrev = (OS_TCB *)0;

+    if (OSRdyTCBQueueNumTbl[grp] == 0) {
+        OSRdyTCBQueueFrontTbl[grp] = ptcb;
+        OSRdyTCBQueueRearTbl[grp]  = ptcb;
+    } else {
+        ptcb->OSRdyTCBPrev                      =
    OSRdyTCBQueueRearTbl[grp];
+        OSRdyTCBQueueRearTbl[grp]->OSRdyTCBNext = ptcb;
+        OSRdyTCBQueueRearTbl[grp]               = ptcb;
+    }
+    OSRdyTCBQueueNumTbl[grp]++;
 }
```

Listing 17: os_core.c —按组入队

3.3.3 OSRdyQueueOut——按组出队

签名改为接受组号参数 grp，逻辑与任务一类似：

```
-OS_TCB *OSRdyQueueOut(void)
+OS_TCB *OSRdyQueueOut(INT8U grp)
 {
     OS_TCB *ptcb;
+    if (OSRdyTCBQueueFrontTbl[grp] == (OS_TCB *)0)
         return (OS_TCB *)0;
+    ptcb = OSRdyTCBQueueFrontTbl[grp];
+    if (OSRdyTCBQueueNumTbl[grp] == 1) {
+        OSRdyTCBQueueFrontTbl[grp] = (OS_TCB *)0;
+        OSRdyTCBQueueRearTbl[grp]  = (OS_TCB *)0;
+    } else {
+        OSRdyTCBQueueFrontTbl[grp]               = ptcb->
    OSRdyTCBNext;
+        OSRdyTCBQueueFrontTbl[grp]->OSRdyTCBPrev = (OS_TCB *)0;
+    }
     ptcb->OSRdyTCBNext = (OS_TCB *)0;
     ptcb->OSRdyTCBPrev = (OS_TCB *)0;
+    OSRdyTCBQueueNumTbl[grp]--;
     return ptcb;
 }
```

Listing 18: os_core.c —按组出队（签名变更）

3.4 调度器修改（OS_SchedNew）

组间优先级调度通过 OSRdyGrp/OSRdyTbl 位图确定最高优先级组号 y，再在该组内进行轮转：

```
 #if OS_SCHED_ROUND_ROBIN_EN > 0
+    if (OSTCBCur == 0)
+    {
+        INT8U y;
+        y = OSUnMapTbl[OSRdyGrp];          /* 最高优先级组号 */
+        OSPrioHighRdy = (INT8U)((y << 3) + OSUnMapTbl[OSRdyTbl[y]]);
+    }
+    else if (OSTCBCur->quantum == 0)
+    {
+        INT8U y;
+        y = OSUnMapTbl[OSRdyGrp];          /* 当前最高优先级组号 */
+
+        if (OSTCBCur->OSTCBPrio != OS_TASK_IDLE_PRIO)
+        {
+            OSRdyQueueOut(OSTCBCur->OSTCBY);  /* 从本组队列移走当前任务 */
+            OSRdyQueueIn(OSTCBCur);           /* 重新入本组队尾 */
+            /* 从最高优先级组的队头取下一个任务 */
+            OSPrioHighRdy = OSRdyTCBQueueFrontTbl[y]->OSTCBPrio;
+        }
+        else
+        {
+            /* 空闲任务：回退到位图调度 */
+            OSPrioHighRdy = (INT8U)((y << 3) + OSUnMapTbl[OSRdyTbl[y]]);
+        }
+    }
 #endif
```

Listing 19: os_core.c —分组轮转调度逻辑

关键区别在于出队时传入的是当前任务所在的组（OSTCBCur->OSTCBY），而选取下一任务时使用的是当前最高优先级组（y）——这两者在组间出现更高优先级任务时会不同，从而保证组间抢占正确工作。

3.5 任务删除适配（os_task.c）

由于 OSRdyQueueOut 的签名已改为接受组号，删除任务时需传入被删除任务的组号：

```
 #if OS_SCHED_ROUND_ROBIN_EN > 0
-    OSRdyQueueOut();
+    OSRdyQueueOut(ptcb->OSTCBY);  /* 按组号出队 */
     OSTCBCur = 0;
 #endif
```

Listing 20: os_task.c —任务删除出队（传入组号）

3.6 测试用例扩展（main.c）

任务二新增 Task 4 和 Task 5，分配到不同的优先级组，以验证组间优先级抢占与组内轮转能够同时正确工作。

```
+static OS_STK task4_stk[TASK1_STK_SIZE];
+static OS_STK task5_stk[TASK1_STK_SIZE];
 void task4(void *p_arg);
 void task5(void *p_arg);

     OSTaskCreate(task3, (void *)0, &task3_stk[...], 9);   /* 组 1 */
+    OSTaskCreate(task4, (void *)0, &task4_stk[...], 11);  /* 组 1, 与 task3 轮转 */
+    OSTaskCreate(task5, (void *)0, &task5_stk[...], 22);  /* 组 2, 优先级低于组 1 */
```

Listing 21: main.c —任务二测试任务（5 个任务，跨组）

其中 Task 1（prio = TASK1_PRIO）和 Task 2（prio = TASK2_PRIO）若在同一组内，则两者轮转；Task 3 与 Task 4 均在第 1 组（优先级 9 和 11），两者也轮转；Task 5 在第 2 组（优先级 22），只有高优先级组任务全部退出后才会被调度，用于验证组间抢占。

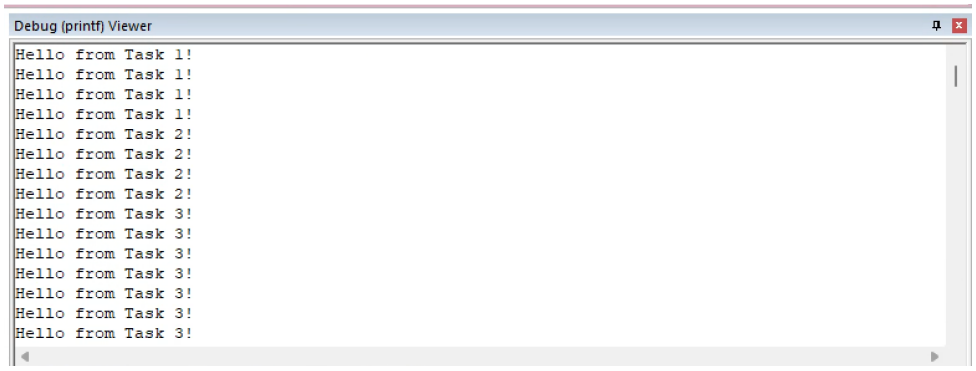
Task 5 设计为无限循环以观察低优先级任务在更高优先级组任务退出后才获得 CPU：

```
+static void task5(void *p_arg)
+{
```

```
+     int i;
+     for (;;)
+     {
+         for (i = 1; i < 10; i++)
+             ;
+         syscall_print_str("Hello from Task 5!\n");
+     }
+}
```

Listing 22: main.c —task5 无限循环（低优先级验证）

经调试，调度器正确工作，在 task1,task2 都退出后，正确的找到了优先级最高的组，并正确的轮换了组里的 task：



4 总结

本次实验在 C/OS-II 的基础上实现了两个层次的时间片轮转调度扩展：

- **任务一**将全体就绪任务组织为一条 FIFO 双向链表，每个时钟中断递减当前任务剩余时间片，时间片耗尽后执行”先出后入”的轮转操作，并修改 `OS_SchedNew` 从队头而非位图选取下一任务。同时扩展了 `OSTaskDel` 的系统调用路径，使非特权模式下的任务能够主动退出并安全地将自身从队列中移除。
- **任务二**将单条队列扩展为按优先级组索引的 8 条队列，在不影响组间优先级抢占语义的前提下，让同一组内的任务轮流共享 CPU，是对任务一数据结构和调度逻辑的自然推广。

两次改动均以条件编译宏 `OS_SCHED_ROUND_ROBIN_EN` 控制，对原有优先级调度路径无侵入。